

TP3-Encapsulation

Exercice 1 : (Livres)

Soit la classe Livre suivante :

```
public class Livre {  
    private String titre,auteur;  
    private int nbPages;  
    //Constructeur  
    public Livre(String unAuteur,String unTitre) {  
  
        titre = unTitre;  
        auteur = unAuteur;  
    }  
    //Accesseur  
    public String getAuteur() {  
        return auteur;  
    }  
    //Modificateur  
    public void setNbPages(int nbPages) {  
        this.nbPages = nbPages;  
    }  
}
```

1. Créer une deuxième classe **TestLivres** qui contiendra la méthode main.
2. Créer les deux livres suivants :
 - livre1 : « Thinking in Java » de Bruce Eckel
 - livre2 : « Programmer en Java » de Claude Delannoy
3. Afficher les auteurs de ces deux livres.
4. Ajouter les accesseurs (getters) pour les attributs titre et nbPages.
5. Ajouter les modificateurs (setters) pour les attributs auteur et titre.
6. Changer le modificateur de nbPages : sa valeur doit être strictement positive.
7. Dans la méthode main :
 - Affecter à chaque livre son nombre de pages.
 - Afficher les deux livres.
 - Calculez le nombre total de pages de ces 2 livres et affichez-le.
8. Dans la classe Livre, ajoutez une méthode *afficher()* qui affiche une description du livre (auteur, titre et nombre de pages). Utilisez *afficher()* dans la méthode main de TestLivres.
9. Ajouter l'instruction `System.out.println(livre1)` où livre désigne un des livres que vous avez créés.
10. Ajouter une méthode *toString()* qui retourne une chaîne de caractères qui décrit le livre. Exécutez à nouveau la classe TestLivres. Remarquez ce qui est affiché maintenant par l'instruction `System.out.println(livre)`.
11. Modifier la méthode *afficher()* pour utiliser *toString()*.

12. Enlever (commenter provisoirement) le seul constructeur de la classe Livre. Sans ajouter de nouveau constructeur, peut-on quand même créer un nouveau livre dans la méthode main ?
13. Remettez le constructeur que vous avez enlevé. Est-ce que le code de la méthode main de la question précédente fonctionne toujours (tester) ?
14. Ajoutez 2 constructeurs pour avoir 3 constructeurs dans la classe :
 - un constructeur qui n'a pas de paramètre,
 - un qui prend en paramètre l'auteur et le titre du livre,
 - et l'autre qui prend en plus le nombre de pages.
15. Utilisez les 3 constructeurs (et éventuellement d'autres méthodes) pour créer 3 nouveaux livres de votre choix dans la méthode main de TestLivres.
16. Dans la méthode main, créer un nouveau livre identique (même valeurs d'attributs) à un livre déjà créé, puis tester l'égalité des deux livres avec l'opérateur ==. Que remarquez-vous ? Que faut-il faire pour tester l'égalité de deux objets ?
17. Ajouter une méthode equals() qui compare deux livres.
18. Ajouter un attribut nbLivres désignant le nombre de livres créés. Quelle est la spécificité de cet attribut ? Où doit-on initialiser la valeur de cet attribut ?
19. Modifier les constructeurs afin de mettre à jour la valeur de l'attribut nbLivres.
20. Ajouter une méthode getNbLivres() permettant de renvoyer le nombre de livres créés. Quel est le type de cette méthode ?
21. Dans la méthode main, afficher le nombre de livres créés après chaque instantiation d'un livre.
22. Mettez la classe Livre dans un package que vous nommez librairie.
23. Résoudre les éventuels problèmes d'importation de la classe Livre dans la classe TestLivres.

Exercice 2 : (Compte Bancaire)

Dans cet exercice, nous allons modéliser le fonctionnement d'une banque. Cela nous permet de faire interagir plusieurs types d'objet différents : le compte et le client.

Comme d'habitude, vous écrirez chaque classe dans un fichier différent. Vous écrirez également un programme principal, ne possédant que la méthode main, dans laquelle vous testerez chacune de vos classes au fur et à mesure de leur écriture.

Les classe compte et client doivent avoir 2 constructeurs un constructeur sans paramètre et un constructeur paramétré.

1. La classe Compte :

La première étape est d'écrire une classe Compte. Celle-ci doit contenir deux attributs privés :

- Son numéro de type int et son solde de type double, et une constante statique le nombre maximal de compte ainsi que quatre méthodes publiques :
- void depot(double valeur); /* pour faire un dépôt sur le compte. */

- void retrait(double valeur); /* pour faire un retrait sur le compte. */
- double getSolde(); /* pour obtenir la valeur du solde */
- void afficherSolde(); /* pour afficher le solde */
- void setSolde(double s) ;/* pour initialiser le solde à la valeur de s mais avant de faire cette instruction on doit vérifier que s est positive * /
- String toString() /*récupérer les informations du compte*/

1.2: Pour faire un virement

Nous allons ajouter une méthode a la classe compte qui aura pour effet de faire un virement vers un autre compte. Cette méthode aura donc deux arguments: la somme à déplacer, et le compte destinataire: void virer(float valeur, Compte destinataire)

1.3: attribut et méthode de classe

-Ajouter à la classe compte un attribut de classe publique pour calculer le nombre de compte construit.

-Ajouter une méthode de classe pour tester qu'elle compte a plus de solde.

Public static Compte plusSolde(Compte c1,Compte c2)

1.4: égalité entre deux comptes

-Ajouter la méthode equal pour tester l'égalité entre le compte c passer en paramètre et le compte courant boolean equals(Object o) ;

2. La classe Client

-Jusque-là, les attributs des objets que nous avons étudiés était des types primitifs (int, float...). Il est également possible d'utiliser un objet comme attribut d'un autre objet.

Ici, chaque client possède un compte. Chaque objet client doit donc avoir un objet compte comme attribut. Vous devez maintenant écrire la classe Client. La classe client doit posséder deux attributs privés :

- private Compte compte ,
- nom.

Elle doit également posséder trois méthodes :

- getNom, qui renvoie le nom du client,
- getSolde, qui renvoie le solde total du client (c'est à dire le solde de son unique compte),

- afficherSolde, qui affiche le solde du client.
- La méthode String toString() et la méthode equals(Object o)

2.1: Plusieurs comptes pour un client

-Nous allons maintenant permettre à chaque client d'avoir plusieurs comptes. Pour cela, au lieu d'avoir un attribut compte, nous allons avoir un attribut comptes, qui sera un tableau contenant plusieurs comptes.

-Par exemple, pour déclarer l'attribut comptes, on peut écrire :

```
public Compte [] comptes=new Compte [100];
```

-Notez que la valeur 100 correspond à la taille du tableau, c'est à dire au nombre maximum de comptes que chaque client peut posséder.

-Pour créer le compte correspondant à la première case du tableau, par exemple, il faut procéder ainsi: comptes[0]=new Compte();

-Modifiez la classe Client selon cette nouvelle organisation. Vous devez ajouter, en plus de l'attribut comptes, l'attribut nbComptes qui contient le nombre de compte du client, une méthode ajouterCompte qui ajoute un nouveau compte. Vous devez également modifier la méthode getSolde pour qu'elle renvoie la somme du solde de tous les comptes du client.

3. La classe Banque :

De même qu'un client peut contenir plusieurs comptes, une banque peut contenir plusieurs clients. Créez une classe Banque pour tester votre travail.